Full Batch gradient descent vs Stochastics Gradient Descent vs Mini Batch gradient Descent

Gradient Descent (Key Idea):

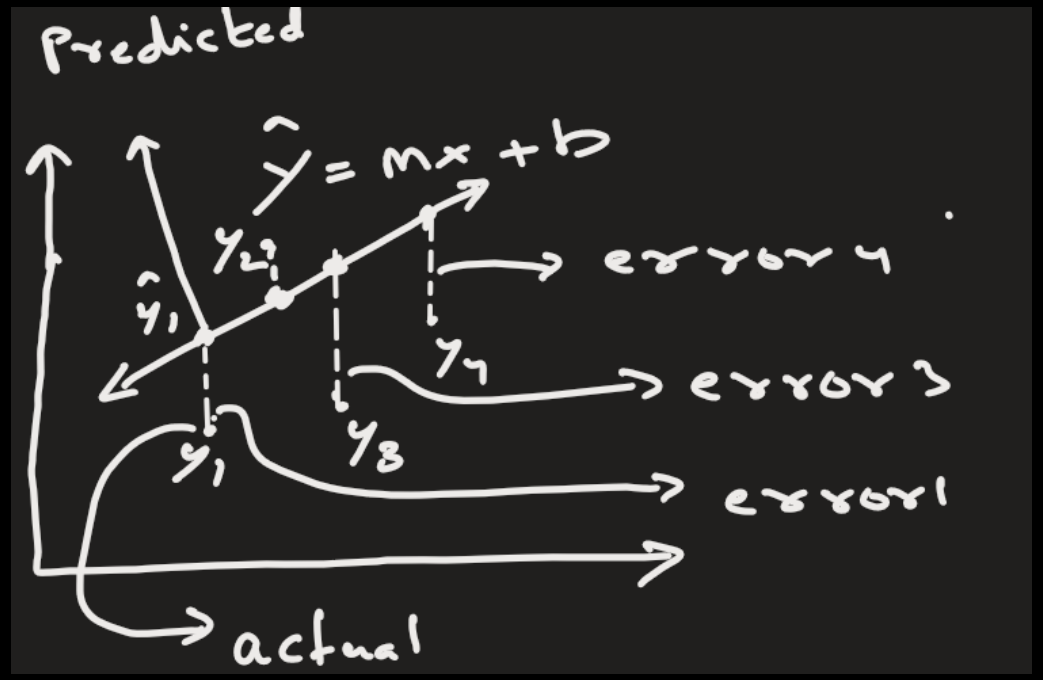
Gradient Descent is an **optimization algorithm** used to:
Minimize a loss (error) function by updating model parameters.

We try to find parameters

$$\theta = (m, b)$$

that minimize error:

$$Loss = \frac{1}{n} \sum (y_{true} - y_{pred})^2$$



The idea:

- Calculate slope (gradient) of loss
- Move parameters in opposite direction of slope
- Repeat until minimum error

Full Gradient Descent (AKA **Batch Gradient Descent)**

Idea: Use **ALL training data** to compute gradient before updating parameters.

Working Algorithm (Step-by-Step)

Assume dataset:

x	y
2	5
5	7
2	8

Step 1 — Initialize parameters

Suppose we are working with linear regression model.

$$y = mx + b$$

It needs m and b.

m = random number

b = random number

learning_rate = α

Step 2 — Loop until convergence

x	y
2	5
5	7
2	8

Step 2.1: Predict for ALL data

$$y_{pred,i} = m x_i + b \quad \text{for all } i$$

Step 2.2: Compute total loss

$$L = \frac{1}{n} \sum (y_i - y_{pred,i})^2$$

Step 2.3: Compute gradients using ALL samples

$$\frac{\partial L}{\partial m} = -\frac{2}{n} \sum x_i (y_i - y_{pred,i})$$

$$\frac{\partial L}{\partial b} = -\frac{2}{n} \sum (y_i - y_{pred,i})$$

Step 2.4: Update parameters ONCE

x	y
2	5
5	7
2	8

$$m = m - \alpha \frac{\partial L}{\partial m}$$

$$b = b - \alpha \frac{\partial L}{\partial b}$$

This is one forward pass. That is, one epoch.

Step 2.5: Repeat

NOTE: Here, one update = one full dataset pass = one epoch

Disadvantages:

- Very slow for large datasets

- High memory usage

x	y
2	5
5	7
2	8

For example:
If we have 100K samples, then **we do forward pass for 100K samples** and compute loss for 100K samples.

If we have 10 features, then we have to compute 100K x 10 = **200K derivatives.**
This is just for one pass.

We do this for many pass. That is **lot of computations**.

Stochastic Gradient Descent (SGD)

x	y
2	5
5	7
2	8

Idea

Instead of using all data, update parameters using **ONE training example at a time**.

Stochastic Gradient Descent (SGD)

x	y
2	5
5	7
2	8

Algorithm

Step 1 — Initialize parameters

m = random

b = random

learning_rate = α

Step 2 — Shuffle dataset

Important to avoid bias.

Stochastic Gradient Descent (SGD)

x	y
2	5
5	7
2	8

Step3 : For EACH sample: For every (x_i, y_i):

Step 3.1: Predict

$$y_{pred} = m x_i + b$$

Step 3.2: Compute loss (single sample)

$$L_i = (y_i - y_{pred})^2$$

Step 3.3: Compute gradient (single point)

$$\frac{\partial L_i}{\partial m} = -2 x_i (y_i - y_{pred})$$

$$\frac{\partial L_i}{\partial b} = 2 (y_i - y_{pred})$$

Stochastic Gradient Descent (SGD)

X1	X2	y
2	3	5
5	4	7
2	5	8

Step 3.4: Update immediately

$$m = m - \alpha \frac{\partial L_i}{\partial m}$$

$$b = b - \alpha \frac{\partial L_i}{\partial b}$$

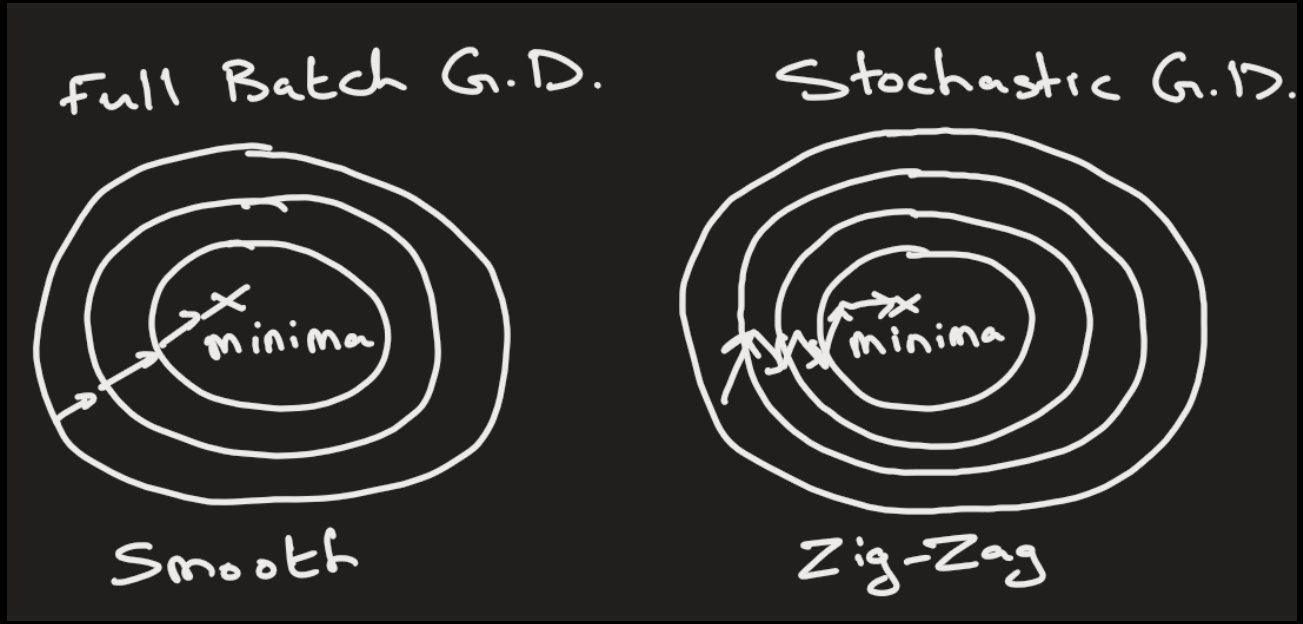
Parameters change after EVERY sample.

Step 4 — Continue next random sample

NOTE: **Many small updates instead of one big update**

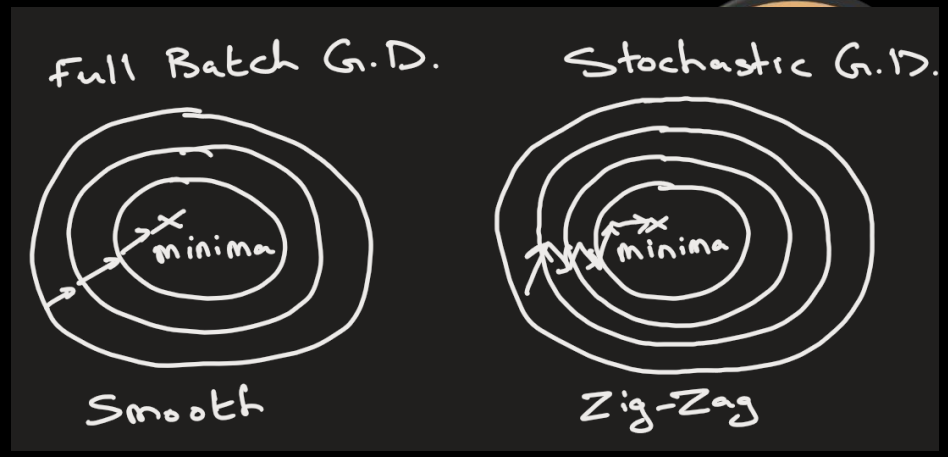
In Full Batch GD, we approach minima smoothly but it is slow.

In Stochastic GD, we approach minima in zig-zag path because we are randomly taking points. But it is fast.



Advantages of SGD

Efficiency: Because it uses only one or a few data points to calculate the gradient, SGD can be much faster, especially for large datasets. Each step requires fewer computations, leading to quicker convergence.



Escaping Local Minima: The noisy updates in SGD, caused by the stochastic nature of the algorithm, can help the model escape local minima or saddle points, potentially leading to better solutions in non-convex optimization problems.

Advantages of SGD

Online Learning: SGD is well-suited for online learning where the model is trained incrementally as new data comes in, rather than on a static dataset.

Memory Efficiency: Since it does not require storing the entire dataset in memory for each iteration, SGD can handle much larger datasets than traditional gradient descent.

Feature	Full Gradient Descent	Stochastic Gradient Descent
Data used	Entire dataset	One sample
Updates	Few	Many
Speed	Slow	Fast
Memory Consumption	High	Low
Large datasets	Poor	Excellent
Escapes local minima	Hard	Easier

Mini-Batch Gradient Descent

x	y
2	5
5	7
2	8

Used in Industry.

Instead of ALL samples or ONE sample, we use **small group of samples (batch)**

Batch size = **32 / 64 / 128 / 256** samples

Mini-Batch Gradient Descent

x	y
2	5
5	7
2	8

Algorithm:

Step 1 — Initialize parameters

m = random
b = random
learning_rate = α
batch_size = k

Suppose we have 1000 samples, and k=10 batch.

Step 2 — Shuffle Dataset

Mini-Batch Gradient Descent

x	y
2	5
5	7
2	8

Step 3 — Split into Mini-Batches

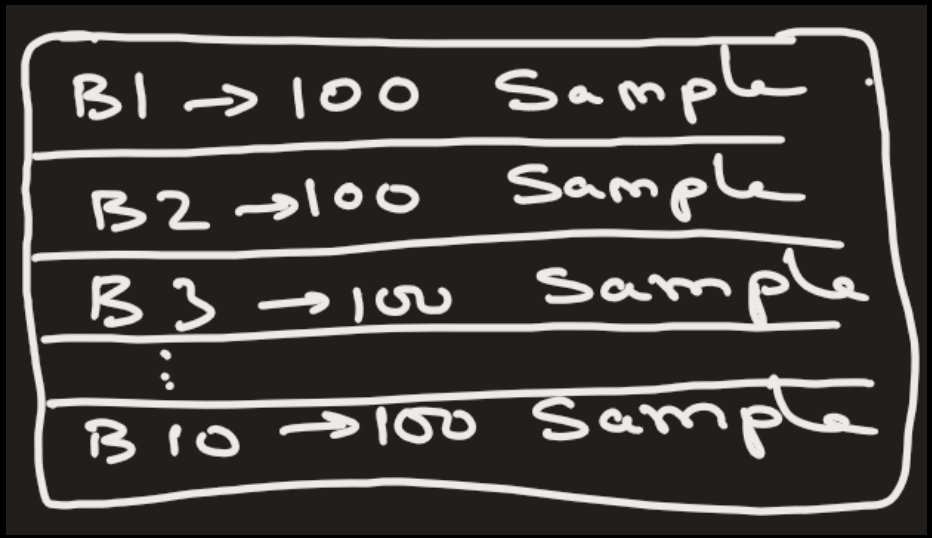
Here we have 1000 samples and 10 batch, so each batch will have 100 samples:

Batch 1 → 100 samples

Batch 2 → 100 samples

Batch 3 → 100 samples

.....



Mini-Batch Gradient Descent

x	y
2	5
5	7
2	8

Step 4 — For EACH mini-batch

Suppose batch contains k samples.

Step 4.1 — Predict for batch

$$y_{pred} = m\, x_{batch} + b$$

(Compute predictions for all k samples)

Step 4.2 — Compute batch loss

$$L_{batch} = \frac{1}{k} \sum (y_i - y_{batch})^2$$

Here, only the batch data is used.

Mini-Batch Gradient Descent

x	y
2	5
5	7
2	8

Step 4.3 — Compute gradients (batch average)

$$\frac{\partial L_{batch}}{\partial m} = -\frac{2}{k} \sum x_i (y_i - y_{pred,i})$$

$$\frac{\partial L_{batch}}{\partial b} = -\frac{2}{k} \sum (y_i - y_{pred,i})$$

Step 4.4 — Update parameters

Update happens after each mini-batch.

$$m = m - \alpha \frac{\partial L_B}{\partial m}$$

$$b = b - \alpha \frac{\partial L_B}{\partial b}$$

Mini-Batch Gradient Descent

x	y
2	5
5	7
2	8

Step 5 — Move to next batch

Repeat until all batches finished → **1 epoch complete**

Mini-Batch Gradient Descent

Batch Size	Effect
Small	Better generalization
Large	Faster computation
Too large	Poor learning

Feature	Batch GD	SGD	Mini-Batch GD
Data used	All	1 sample	Small batch
Speed	Slow	Very fast	Fast
Stability	Very stable	Noisy	Balanced ✅
GPU friendly	Poor	Poor	Excellent ✅
Industry usage	Rare	Rare	**Standard** ⭐

STOP

Why Deep Learning Uses Mini-Batch GD

Neural networks require:

- millions of samples
- GPU parallelism
- frequent updates

Mini-batch enables:

- matrix operations
- GPU acceleration

Challenges of SGD

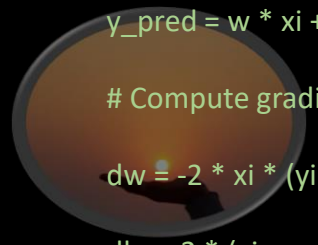
Noisy Convergence: Since the gradient is estimated based on a single data point (or a small batch), the updates can be noisy, causing the cost function to fluctuate rather than steadily decrease. This makes convergence slower and more erratic than in batch gradient descent.

Learning Rate Tuning: SGD is highly sensitive to the choice of learning rate. A learning rate that is too large may cause the algorithm to diverge while one that is too small can slow down convergence. Adaptive methods like Adam and RMSprop address this by adjusting the learning rate dynamically during training.

Long Training Times: While each individual update is fast, the convergence might take a longer time overall since the steps are more erratic compared to batch gradient descent.

Heading Goes Here

Fhdsklf
Fjdsklf
Fjskldf



```
    y_pred = w * xi + b

    # Compute gradients (derivatives)

    dw = -2 * xi * (yi - y_pred)   # derivative wrt w

    db = -2 * (yi - y_pred)        # derivative wrt b


    # Update parameters

    w -= lr * dw

    b -= lr * db




# Compute loss at the end of the epoch

loss = np.mean((y - (w*X + b))**2)


# Check stopping condition

if abs(prev_loss - loss) < tol:

    print(f"Stopped early at epoch {epoch+1}")

    break


prev_loss = loss
```